

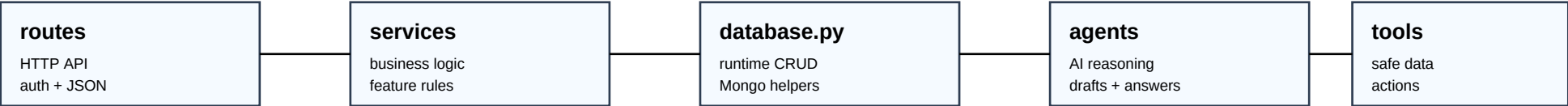
Backend Folder Explanation

RecruitmentAssist backend structure and responsibilities

Backend architecture in one sentence

The backend is a Flask application split into routes, services, database helpers, agentic workflows, tools, and safety layers so normal HR features and AI assistant features can share the same data safely.

How the main layers connect



- Routes are the API boundary.
- Services hold normal product behavior.
- Agents, workflows, tools, schemas, and security support the recruiter assistant and agentic screening flow.
- Root files like app.py and database.py wire the application together.

Backend Folders - Part 1

Each folder's purpose and common files

agents	AI role modules Contains specialized agent logic: router, JD, resume, matching, ranking, summary, interview, and recruiter chat agents. These modules decide or generate intelligent responses, but still rely on tools/services for data and side effects. Key files: router_agent.py, recruiter_agent.py, resume_agent.py, matching_agent.py
app	Core extraction and matching helpers Older/core application utilities used by services: LLM extraction, regex fallback extraction, text parsing, screening summaries, matching logic, and cleaning helpers. Key files: llm_extraction.py, regex_extractor.py, text_extractor.py, matcher.py
database	Database setup and migration scripts Operational scripts and SQL files for schema setup, migration, Mongo import, upload import, seed data, and admin password fixes. Runtime database helpers live in backend/database.py. Key files: schema.sql, migrate_json_to_db.py, seed_mongo_data.py
orchestration	Agent routing and graph coordination Coordinates agentic requests. The main orchestrator tracks runs, hr_graph routes tasks through a graph of nodes, and tracing records readable execution events. Key files: main_orchestrator.py, hr_graph.py, tracing.py

Backend Folders - Part 2

Each folder's purpose and common files

routes	Flask API endpoints HTTP boundary for the backend. Routes receive frontend requests, validate/authenticate them, call services or workflows, and return JSON responses. Key files: candidate_routes.py, jd_routes.py, interview_routes.py, agentic_routes.py
schemas	Request validation models Structured request validation for agentic payloads. It keeps chatbot/orchestrator inputs predictable before they reach agents and tools. Key files: agentic.py
security	Agent and tool safety controls Safety rules for context size, prompt-injection detection, and tool policy. This folder helps prevent unsafe assistant behavior and unconfirmed side effects. Key files: context_safety.py, policy.py
services	Business logic layer Feature logic used by routes: authentication, candidates, JDs, matching, interviews, and reports. Services transform data and enforce app-level behavior. Key files: candidate_service.py, interview_service.py, matching_service.py
static	Served static files and uploads Stores static assets served by Flask, such as the ShimentoX logo and uploaded resume files. Upload files are data artifacts, not application logic. Key files: ShimentoX logo, uploads/

Backend Folders - Part 3

Each folder's purpose and common files

templates	Flask template placeholder Reserved for server-rendered templates. In this project the React frontend is primary, so this folder mostly exists for Flask compatibility. Key files: .gitkeep
tests	Automated backend checks Backend tests for safety and important behavior. These provide regression checks for security hardening and can be expanded as the backend grows. Key files: test_security_hardening.py
tools	Safe helper capabilities for agents Small focused helpers that agents can call for app data or approved actions. The gateway centralizes confirmation/policy checks for side-effecting tools. Key files: recruiter_tools.py, matching_tools.py, tool_gateway.py
workflows	Multi-step application flows Orchestrates larger processes like screening and recruiter chat by calling agents, tools, and services in the right order. Key files: app_workflow.py, screening_workflow.py, recruiter_chat_workflow.py

Backend Root Files

Important files directly inside backend/

app.py	Flask application entry point. Registers blueprints, static frontend serving, CORS/session behavior, and API startup wiring.
database.py	Runtime database access layer. Centralizes Mongo/database CRUD helpers used by routes, services, tools, and agents.
requirements.txt	Python dependencies for running the backend.
spa_urls.py	Frontend route allow-list/helper for serving the React single-page application.
.env / .env.example	Local configuration and example environment variables for database, SMTP, OpenAI, and app secrets.

Folders usually ignored while explaining code

__pycache__ is Python bytecode cache generated automatically. static/uploads can contain user-uploaded resumes and should be treated as runtime data rather than source code.